

Amazon Bedrock AgentCore Payments

Private Preview

AWS CONFIDENTIAL Private Preview Use Only

What is AgentCore?

[Amazon Bedrock AgentCore](#) is a fully managed platform for building, deploying, and operating AI agents at scale. It provides a comprehensive set of capabilities including Runtime, Memory, Gateway, Identity, Code Interpreter, Browser, Observability, Evaluations, and Policy so developers can focus on agent logic rather than undifferentiated infrastructure. AgentCore works with popular opensource frameworks (CrewAI, LangGraph, LlamaIndex, [Strands Agents](#), and more) and any foundation model.

Amazon Bedrock AgentCore Runtime provides a secure, serverless and purpose-built hosting environment for deploying and running AI agents or tools. It offers the following benefits like being framework agnostic, Model flexibility, Protocol support, Session isolation, Extended execution time, Consumption-based pricing model, Built-in authentication, Agent-specific observability, Unified set of agent-specific capabilities and more. To host an agent on AgentCore runtime follow [this](#) guide here for all the detailed steps.

- 1) Install the libraries and setup your environment
- 2) Use the *agentcore create* command to set up a skeleton agent project with the framework of your choice.
- 3) Host your agent in AgentCore Runtime using *agentcore launch*
- 4) Test your deployed agent using *agentcore invoke* command

For more information, please refer to these [tutorials](#) in python.

What is AgentCore Payments?

AgentCore Payments is a new Amazon Bedrock AgentCore service that provides the most secure, easiest, and most flexible way to enable microtransaction payments in AI agents to access paid APIs, MCP servers, and content.

AI agents increasingly handle complex tasks by calling APIs, accessing MCP servers, and interacting with other agents. As more services monetize through pay-per-use models, developers face significant challenges integrating payments into agentic workflows. Transactions are typically microtransactions often under \$1 or fractions of a cent making traditional payment methods cost-prohibitive due to high minimum transaction fees (\$0.30+). Meanwhile, content providers are introducing payment-based access for AI agents.

AgentCore Payments eliminates this friction with a suite of developer-friendly capabilities that enable secure instant payments to paid services with stablecoin support and open protocols like x402 for cost-effective microtransactions and configurable guardrails to control agent spending reducing developer effort from months to days.

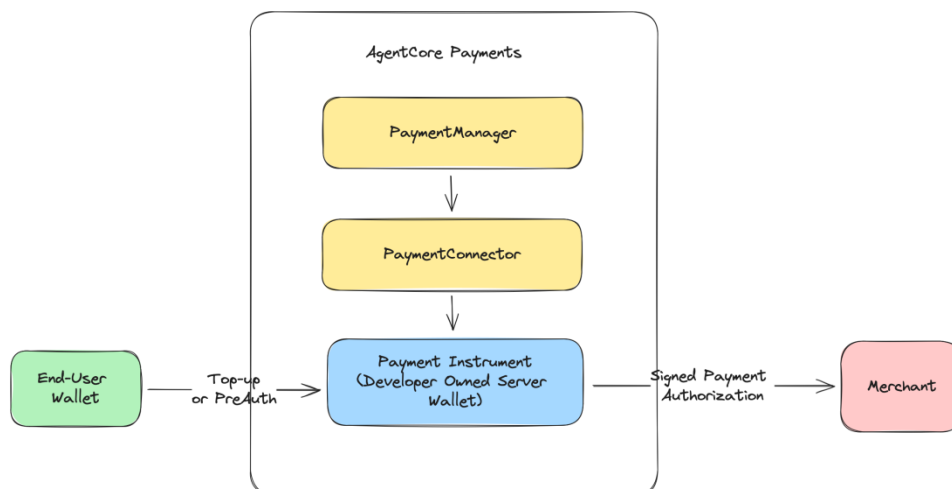
Core Capabilities

- **Payment Connector:** Enables end users to connect to their third-party digital wallet by capturing access credentials to retrieve and store payment tokens associated with their payment method to facilitate agentic transactions.
- **Payment Manager:** A unified execution engine that manages the complete payment flow from wallet initiation through merchant payments, supports multi-step payment interactions, and handles exceptions such as declines and retries.
- **Secure Wallet Authentication:** Leverages AgentCore Identity to securely store private third-party wallet-specific authentication keys (e.g., Wallet ID) and authenticate agents and users with their wallets.
- **Spending Guardrails:** Allows developers to define user-specific and agent-specific spending limits at the session level and enforce limits during runtime.
- **Discoverability:** Offers ready-to-use MCP servers like Coinbase Bazaar with pay-per-use x402 endpoints. Agent developers can discover and search for relevant endpoints for their use cases.
- **End-to-End Observability:** Leverages AgentCore Observability to provide comprehensive visibility across the payment lifecycle with detailed logs, dashboards, and metrics to monitor transaction success, failures, and overall system health.

Personas

- **Agent Developers:** Build AI agents that need to execute microtransaction payments to access paid APIs, MCP servers, and web content. Agent developers also handle administrative tasks such as configuring spending guardrails, monitoring payment activity, and managing wallet credential policies. AgentCore Payments reduces their implementation effort from 2–3 months to days.
- **End Users:** Users of the agent to achieve a certain goal. They can top up agent balance so that agents can execute transactions on their behalf, with configurable spending limits and full transparency.
- **Merchants:** API providers, content providers, and service operators who monetize their offerings through pay-per-use models. Merchants are the counterparties that agents interact with when making payments they set pricing, accept x402 payments, and deliver the requested content or service upon payment confirmation.

Scope for Private Preview



This private preview release includes:

- **Payment Manager management (create, get, list, update, delete):** A Payment Manager is the top-level resource that represents your payment processing configuration. It ties together your IAM authorization, connectors, and instruments into a single logical unit.
- **Payment Connector management (create, get, list, update, delete):** A Payment Connector links a credential provider (e.g., Coinbase CDP) to a Payment Manager, enabling the manager to interact with a specific payment network or wallet provider. (Coinbase Developer Platform supported in private preview).
- **Payment Credential Provider management (create, get, list, update, delete):** A Credential Provider securely stores your third-party wallet credentials (API keys, secrets) using AgentCore Identity, so the service can authenticate with external wallet providers on your behalf at runtime. (Coinbase Developer Platform supported in private preview).
- **Payment Instrument management (create, get, list):** A Payment Instrument represents an end user's payment method (e.g., a crypto wallet). When created, the service provisions a wallet on the configured network that the user can fund and use for transactions. (USDC stablecoin supported for private preview on Base and Solana).
- **Payment Session management (create, get, list):** A Payment Session is a time-bound context for a set of transactions with configurable spending limits. Sessions enforce budgeting guardrails so agents cannot exceed the approved spend for a given task or workflow. Budget is an optional field.
- **Discovery via MCP:** Connection to an MCP server endpoint which has x402 pay-per-use endpoints listed in Coinbase x402 Bazaar. Users or their agents can discover and search for relevant endpoints for their use case.
- **Payment processing with x402 protocol (crypto microtransactions via stablecoin/USDC):** The core transaction capability. When an agent encounters a paid endpoint using the x402 protocol, it submits the payment requirement to ProcessPayment, which signs and authorizes the transaction using the user's instrument within the session's budget. It currently supports both x402 versions V1 and V2.

Testing Instructions

Prerequisites

1. Account Allowlisting

Your AWS account must be allowlisted for the AgentCore Payments preview endpoints. Contact your AWS representative if your account has not been allowlisted.

2. IAM Role Configuration

The quickstart scripts from the sample code shared contains *quickstart/setup_roles.sh* to create four IAM roles automatically. If you prefer to create them manually, here is the role structure:

Role	Permissions	Purpose
ControlPlaneRole	bedrock-agentcore:* (except ProcessPayment), secretsmanager:CreateSecret/PutSecretValue, iam:PassRole	Control plane setup: create managers, connectors, credential providers
ManagementRole	Instrument/session CRUD. Deny ProcessPayment.	Application backend / human UI: create wallets, set budgets
ProcessPaymentRole	bedrock-agentcore:ProcessPayment only	Agent execution: can only spend within the budget set by the application
ResourceRetrievalRole	Token-vault, workload-identity, SecretsManager (read), sts:SetContext	Service role assumed by AgentCore Payments at runtime

Why separate roles? The role separation enforces a security boundary between the application backend and the agent. The application backend (ManagementRole) creates instruments and sessions with spending limits. The agent (ProcessPaymentRole) can only execute payments within the budget it was given it cannot create sessions, override limits, or provision new wallets. This ensures an agent cannot exhaust wallet funds by creating unlimited sessions.

The **ResourceRetrievalRole** requires the following trust relationship (for the AgentCore service to assume it at runtime):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAccessToBedrockAgentcore",
      "Effect": "Allow",
      "Principal": {
        "Service": "bedrock-agentcore.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    },
  ],
}
```

```

    "Sid": "AllowPreviewAccessToBedrockAgentcore",
    "Effect": "Allow",
    "Principal": {
      "Service": "preprod.genesis-service.aws.internal"
    },
    "Action": "sts:AssumeRole"
  }
]
}

```

The other three roles (ControlPlane, Management, ProcessPayment) should have a trust policy allowing your account to assume them.

Note: Save all role ARNs you will need them for setup and testing.

3. Coinbase CDP API Keys and Wallet Funding

AgentCore Payments uses [Coinbase Developer Platform \(CDP\)](#) as the wallet provider during the preview. You need CDP API credentials and a funded testnet wallet.

Step 1: Create a CDP API key

1. Sign up or log in at <https://portal.cdp.coinbase.com/>
2. Navigate to **API Keys** and create a new key
3. Save the three values you will need them during quickstart setup:
 - API Key ID
 - API Key Secret
 - Wallet Secret

Security: These credentials grant access to wallet operations. Store them securely and never commit them to source control. The quickstart .env file is git-ignored by default.

Step 2: Fund your testnet wallet

After running the quickstart (which creates a wallet via CreatePaymentInstrument), fund it with testnet USDC:

1. Copy the walletAddress from the CreatePaymentInstrument response
2. Go to the Circle USDC faucet: <https://faucet.circle.com/>
3. Select **Base Sepolia** as the network
4. Paste your wallet address and request USDC

The faucet provides free testnet USDC no real money is involved. You can verify the balance on Base Sepolia Explorer (<https://sepolia.basescan.org/>) by searching for your wallet address.

Note: All testing uses the Base Sepolia testnet (eip155:84532). Do not use mainnet wallets or real cryptocurrency.

API Overview

Control Plane APIs

API	Role	Description
CreatePaymentCredentialProvider	ControlPlaneRole	Register a third-party wallet credential provider (e.g., Coinbase CDP)
CreatePaymentManager	ControlPlaneRole	Create a payment manager with IAM authorization
CreatePaymentConnector	ControlPlaneRole	Link a credential provider to a payment manager

Data Plane APIs

API	Role	Description
CreatePaymentInstrument	ManagementRole	Create a payment instrument (e.g., crypto wallet) for a user
GetPaymentInstrument	ManagementRole	Retrieve details of a payment instrument
ListPaymentInstruments	ManagementRole	List all payment instruments for a user
CreatePaymentSession	ManagementRole	Create a time-bound payment session with optional spending limits
GetPaymentSession	ManagementRole	Retrieve details of a payment session
ListPaymentSessions	ManagementRole	List all payment sessions for a user
ProcessPayment	ProcessPaymentRole	Signs and initiate a payment transaction using the x402 protocol

Please refer to the full API specification provided for more details.

SDK Client Setup

AgentCore Payments uses two service clients:

Client	Service Name	Purpose
Control Plane	bedrock-agentcore-control	Credential providers, managers, connectors
Data Plane	bedrock-agentcore	Instruments, sessions, payments

Private Preview Endpoints

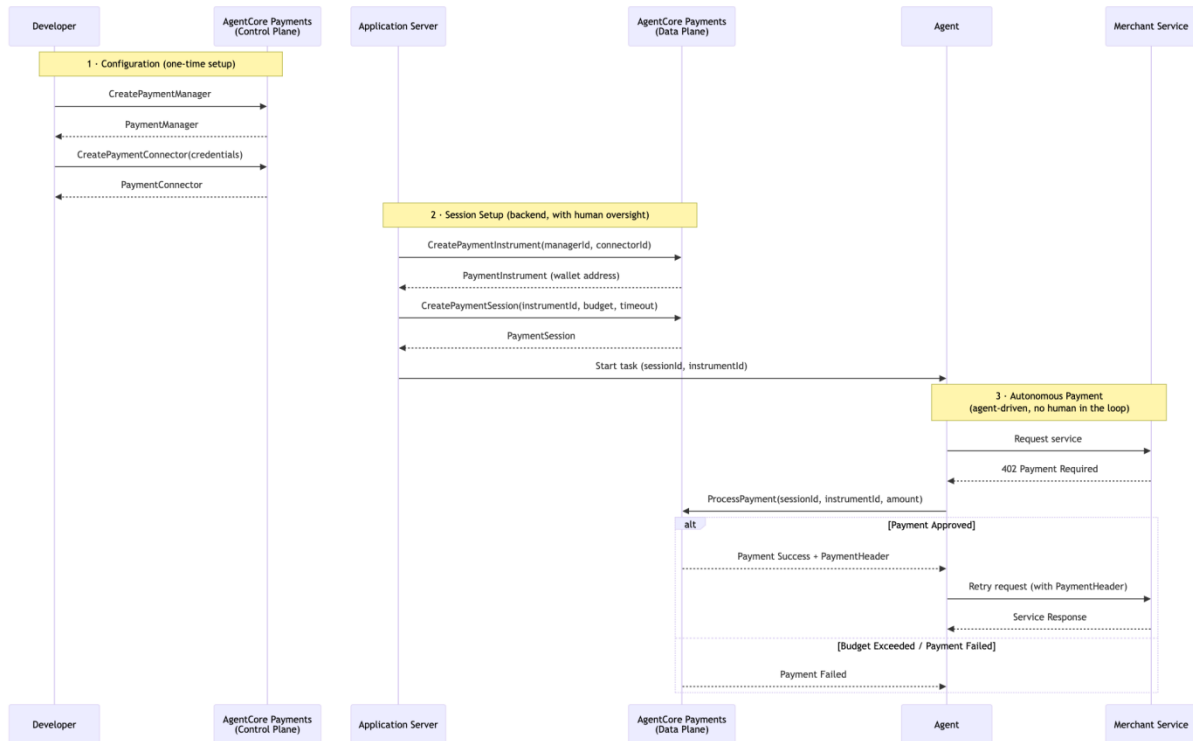
```
For us-west-2 region:
# Control Plane + Credential Provider
export CP_ENDPOINT=bedrock-agentcore-control.us-west-2.amazonaws.com
# Data Plane
export DP_ENDPOINT=bedrock-agentcore.us-west-2.amazonaws.com

For us-east-1 region:
# Control Plane + Credential Provider
export CP_ENDPOINT=bedrock-agentcore-control.us-east-1.amazonaws.com
# Data Plane
export DP_ENDPOINT=bedrock-agentcore.us-east-1.amazonaws.com
```

End to End Walkthrough

Before starting, ensure your AWS credentials are configured and active. You can use any standard method AWS CLI profiles, environment variables, or your organization's credential management tool.

Sequence Diagram



Prerequisites

- Python 3.9+
- boto3 (stock PyPI — no preview wheels)
- AWS credentials configured (aws configure or environment variables)

Verify with:

```
aws sts get-caller-identity
```

- Service model files (provided in /quickstart/model/ directory)

```
mkdir -p ~/.aws/models/bedrock-agentcore-control/2023-06-05
mkdir -p ~/.aws/models/bedrock-agentcore/2024-02-28

# Copy the .normal.json as service-2.json from the quickstart:
cp ~/.aws/models/bedrock-agentcore-control/2023-06-05/normal.json \
  ~/.aws/models/bedrock-agentcore-control/2023-06-05/service-2.json
cp ~/.aws/models/bedrock-agentcore/2024-02-28/normal.json ~/.aws/models/bedrock-agentcore/2024-02-28/service-2.json
```

- Roles are setup correctly using the setup_roles.sh script
- Install required libraries `pip install boto3 python-dotenv`

```

import boto3
import os
# Path to the service model files
MODELS_DIR = os.path.join(os.path.dirname(__file__), "models")
# Create a session and register the custom service models
session = boto3.Session(region_name="us-west-2")
session._session.get_component("data_loader").search_paths.insert(0,
MODELS_DIR)
# Control Plane client
cp = session.client(
    "bedrock-agentcore-control",
    endpoint_url="https://bedrock-agentcore-control.us-west-2.amazonaws.com",
)
# Data Plane client
dp = session.client(
    "bedrock-agentcore",
    endpoint_url="https://bedrock-agentcore.us-west-2.amazonaws.com",
)

```

Control Plane Setup (ControlPlaneRole)

These steps are handled automatically by quickstart/setup_roles.sh. The examples below are for manual testing.

Step 1: Create Payment Credential Provider

Register your third-party wallet credentials (e.g., Coinbase CDP) and store your API Key, Secret and WalletSecret securely in your environment variables.

Create a Credential Provider

Stores your payment partner credentials (e.g. Coinbase CDP API keys) securely in AgentCore.

```

import os

resp = cp.create_payment_credential_provider(
    name="MyCoinbaseProvider",
    credentialProviderVendor="CoinbaseCDP",
    providerConfigurationInput={
        "coinbaseCdpConfiguration": {
            "apiKeyId": os.environ["COINBASE_API_KEY_ID"],
            "apiKeySecret": os.environ["COINBASE_API_KEY_SECRET"],
            "walletSecret": os.environ["COINBASE_WALLET_SECRET"],
        }
    },
)

credential_provider_arn = resp["credentialProviderArn"]
print(f"Credential Provider ARN: {credential_provider_arn}")

```

Save: Note the credentialProviderArn from the response you will need it in Step 3.

Get a Credential Provider

```

resp = cp.get_payment_credential_provider(name="MyCoinbaseProvider")
print(resp["credentialProviderArn"])
print(resp["status"])

```

List Credential Providers

```
resp = cp.list_payment_credential_providers(maxResults=20)
for provider in resp["credentialProviders"]:
    print(f"    {provider['name']} - {provider['credentialProviderArn']}")
```

Delete a Credential Provider

```
cp.delete_payment_credential_provider(name="MyCoinbaseProvider")
```

Step 2: Create Payment Manager

Create a payment manager linked to the ResourceRetrievalRole:

```
import uuid

resp = cp.create_payment_manager(
    name="MyManager",          # [a-zA-Z][a-zA-Z0-9_]{0,47}
    description="Production manager", # [a-zA-Z0-9\s]+
    authorizerType="AWS_IAM",
    roleArn="arn:aws:iam::<ACCOUNT_ID>:role/<YOUR_ROLE>",
    clientToken=str(uuid.uuid4()) + "-" + str(uuid.uuid4())[:8], # >= 33 chars
)

manager_id = resp["paymentManagerId"] # short ID - used in CP calls
manager_arn = resp["paymentManagerArn"] # full ARN - used in DP calls
print(f"Manager ID: {manager_id}")
print(f"Manager ARN: {manager_arn}")
```

Note: Client token is optional but if provided we will ensure idempotency and we encourage you to send it to create and process payment APIs.

Get a Payment Manager

```
resp = cp.get_payment_manager(paymentManagerId=manager_id)
print(resp["name"], resp["status"])
```

List Payment Managers

```
resp = cp.list_payment_managers(maxResults=100)
for proc in resp["paymentManagers"]:
    print(f"    {proc['name']} - {proc['paymentManagerId']}")
```

Delete a Payment Manager

```
cp.delete_payment_manager(
    paymentManagerId=manager_id,
    clientToken=str(uuid.uuid4()) + "-" + str(uuid.uuid4())[:8],
)
```

Save: Note the paymentManagerArn and paymentManagerId from the response. Data Plane APIs use paymentManagerArn (both in the request body and URL path).

Step 3: Create Payment Connector

Link the credential provider to the payment manager (add manager_id and credential_provider_arn:

```
resp = cp.create_payment_connector(
    paymentManagerId=manager_id,
    name="MyCoinbaseConnector",
```

```

        description="Coinbase CDP connector",    # alphanumeric + spaces only
        type="CoinbaseCDP",
        credentialProviderConfigurations=[{
            "coinbaseCDP": {
                "credentialProviderArn": credential_provider_arn,
            }
        }],
        clientToken=str(uuid.uuid4()) + "-" + str(uuid.uuid4())[:8],
    )

connector_id = resp["paymentConnectorId"]print(f"Connector ID: {connector_id}")

```

List Payment Connectors

```

resp = cp.list_payment_connectors(
    paymentManagerId=manager_id,
    maxResults=100,
)for conn in resp["paymentConnectors"]:
    print(f"    {conn['name']} - {conn['paymentConnectorId']}")

```

Save: Note the paymentConnectorId from the response.

Application Backend Operations (ManagementRole)

These steps represent what your application backend or human UI would do creating wallets for users and setting budgets before handing off to the agent.

Step 4: Create Payment Instrument

Create a crypto wallet instrument for a user via the payment partner. This is typically done when a user signs up or enables the payment feature. All data plane calls use `paymentManagerArn` (the full ARN, not the short ID).

```

resp = dp.create_payment_instrument(
    paymentManagerArn=manager_arn,
    paymentConnectorId=connector_id,
    userId="user-123",
    paymentInstrumentType="CRYPTO_WALLET",
    paymentInstrumentDetails={
        "cryptoWallet": {"network": "ETHEREUM"}
    },
    clientToken=str(uuid.uuid4()) + "-" + str(uuid.uuid4())[:8],
)

instrument = resp["paymentInstrument"]
instrument_id = instrument["paymentInstrumentId"]
wallet_address =
instrument["paymentInstrumentDetails"]["cryptoWallet"]["walletAddress"]print(f"Inst
rument ID: {instrument_id}")print(f"Wallet Address: {wallet_address}")

```

Save: Note the paymentInstrumentId and walletAddress from the response. Fund the wallet with testnet USDC at <https://faucet.circle.com/> (select Base Sepolia).

Notes: To use Solana, specify **"SOLANA"** in the network and fund the wallet with testnet USDC at <https://faucet.circle.com/> (select Solana Devnet) for your testing.

Step 5 (Optional): Verify Payment Instrument

Retrieve the instrument to confirm it was created successfully:

Get a Payment Instrument

```
resp = dp.get_payment_instrument(
    paymentManagerArn=manager_arn,
    paymentConnectorId=connector_id,
    paymentInstrumentId=instrument_id,
    userId="user-123",
)print(resp["paymentInstrument"]["paymentInstrumentDetails"])
```

List Payment Instruments

```
resp = dp.list_payment_instruments(
    paymentManagerArn=manager_arn,
    paymentConnectorId=connector_id,
    userId="user-123",
    maxResults=10,
)for inst in resp["paymentInstruments"]:
    print(f"    {inst['paymentInstrumentId']}")
```

The response should show "status": "ACTIVE" and include the wallet address.

Step 6: Create Payment Session

Create a time-bound session with spending limits (optional). This is typically done before invoking an agentic workflow (e.g., a deep research session), collecting the user's consent on the budget.

```
resp = dp.create_payment_session(
    paymentManagerArn=manager_arn,
    userId="user-123",
    expiryDuration=300, # minutes - minimum 15 mins and max 480 mins.
    limits={
        "maxSpendAmount": {
            "value": "1.0", # string - $1.00 USD
            "currency": "USD",
        }
    },
)

payment_session = resp["paymentSession"]
session_id = payment_session["paymentSessionId"]
print(f"Session ID: {session_id}")
```

Save: Note the paymentSessionId from the response. The application passes this (along with the paymentInstrumentId) to the agent.

Notes: value must be a string ("1.0"), not a number. currency is "USD" (not "USDC"). The service converts USDC to USD for budget enforcement. The agent receives the paymentSessionId and paymentInstrumentId but cannot create new sessions or modify the budget.

Agent Payment Execution (ProcessPaymentRole)

This step represents what the agent does autonomously. The agent receives a `paymentSessionId` and `paymentInstrumentId` from the application backend and can only execute payments within the budget. It should be implemented as an interceptor or hook in the deterministic code path the agent LLM should NOT have direct access to this API.

Step 8: Process Payment (x402)

Execute a microtransaction payment using the x402 protocol. The `cryptoX402` payload object is passed as-is from the paid endpoint's x402 `PaymentRequirement`. The code is for version 2.

```
resp = dp.process_payment(
    paymentManagerArn=manager_arn,
    userId="user-123",
    paymentSessionId=session_id,
    paymentInstrumentId=instrument_id,
    paymentType="CRYPTO_X402",
    paymentInput={
        "cryptoX402": {
            "version": "2",
            "payload": {
                "scheme": "exact",
                "network": "eip155:84532",    # Base Sepolia
                "amount": "100000",          # $0.10USDC
                # USDC contract
                "asset": "0x036CbD53842c5426634e7929541eC2318f3dCF7e",
                # recipient
                "payTo": "<add_merchant_address>",
                "maxTimeoutSeconds": 300,
                "extra": {"name": "USDC", "version": "2"},
            }
        }
    },
    clientToken=str(uuid.uuid4()) + "-" + str(uuid.uuid4())[:8],
)
payment = resp["paymentOutput"]["cryptoX402"]
print(f"Status: {payment['status']}")
# PROOF GENERATED
print(f"Signature: {payment['signature'][:20]}...")
print(f"Authorization: {payment['authorization']}")
```

if you want to use x402 version 1, use the payload below:

```
"payload": {
    "scheme": "exact",
    "network": "base-sepolia",
    "maxAmountRequired": "5000",
    "resource": "https://nickeljoke.vercel.app/api/joke",
    "description": "Premium AI joke generation",
    "mimeType": "application/json",
    "payTo": "<add_merchant_address>",
    "maxTimeoutSeconds": 300,
    "asset": "0x036CbD53842c5426634e7929541eC2318f3dCF7e",
    "outputSchema": {
        "input": {
            "type": "http",
            "method": "GET",
            "discoverable": true
        }
    }
},
```

```

    "extra": {
      "name": "USDC",
      "version": "2"
    }
  }
}

```

A successful response will show "status": "PROOF_GENERATED" with the signed payment authorization.

Notes: All fields including version and extra are required. amount is in the smallest unit of the token (100000 = \$0.10 USDC with 6 decimal places). The ProcessPaymentRole can ONLY call ProcessPayment it cannot create sessions, instruments, or override budgets.

To use Solana for payment, you just need to pass the solana “accept” payload in processPayment call.

Post-Payment Verification (ManagementRole)

The application backend can verify session state after the agent has executed payments.

Step 9 (Optional): Get Payment Session

```

resp = dp.get_payment_session(
    paymentManagerArn=manager_arn,
    paymentSessionId=session_id,
    userId="user-123",
)
session = resp["paymentSession"]
print(f"Status: {session['status']}")
print(f"Spend: {session.get('currentSpendAmount', {})}")

```

List Payment Sessions

```

resp = dp.list_payment_sessions(
    paymentManagerArn=manager_arn,
    userId="user-123",
    maxResults=10,
)
for s in resp["paymentSessions"]:
    print(f"    {s['paymentSessionId']} - {s['status']}")

```

AgentCore observability through Amazon cloudwatch

To help you monitor and troubleshoot your Bedrock AgentCore Payment integration, we support observability through Amazon CloudWatch.

Amazon CloudWatch is AWS's monitoring and observability service. It collects and tracks metrics, logs, and traces from your AWS resources, giving you visibility into how your applications are performing.

Vended logs are logs that AWS services publish on your behalf directly to your CloudWatch Log Group. Unlike application logs you instrument yourself; vended logs are generated automatically by the service — you just configure where they should be delivered.

Spans represent units of work within a request (e.g., an API call and its downstream operations). They help you trace the flow of a request through the system, making it easier to identify latency bottlenecks or failures.

The following section walks through setting up CloudWatch vended logs and spans delivery for Bedrock AgentCore Payment Manager. After completing these steps, any data plane API call (e.g., `CreateInstrument`) will produce logs and trace data visible in your configured CloudWatch Log Group.

Prerequisites

- A CloudWatch Log Group as the delivery destination (e.g., `/bedrock-agentcore/payments/test-vended-logs-gamma`). If you don't have one yet, you can create one from the AWS Console or CLI:

Console: Open the CloudWatch console, go to Logs → Log groups, and choose Create log group. Enter a name like `/bedrock-agentcore/payments/my-logs` and select your preferred retention period.

CLI:

```
aws logs create-log-group --log-group-name /bedrock-agentcore/payments/my-logs
```

- The resource ARN of your Bedrock AgentCore Payment Manager

Permissions needed on role to create log and span resources

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CloudWatchLogsVendedDelivery",
      "Effect": "Allow",
      "Action": [
        "logs:CreateDelivery",
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:DeleteDelivery",
        "logs:DeleteDeliveryDestination",
        "logs:DeleteDeliverySource",
        "logs:DeleteLogGroup",
        "logs:DeleteResourcePolicy",
        "logs:DescribeLogGroups",
        "logs:DescribeResourcePolicies",
        "logs:GetDelivery",
        "logs:GetDeliveryDestination",
        "logs:GetDeliverySource",
        "logs:PutDeliveryDestination",
        "logs:PutDeliverySource",
        "logs:PutLogEvents",
        "logs:PutResourcePolicy",
        "logs:PutRetentionPolicy"
      ],
      "Resource": "*"
    }
  ],
  {
```

```

    "Sid": "XRayApplicationSignalsCloudTrail",
    "Effect": "Allow",
    "Action": [
        "xray:GetTraceSegmentDestination",
        "xray:ListResourcePolicies",
        "xray:PutResourcePolicy",
        "xray:PutTelemetryRecords",
        "xray:PutTraceSegments",
        "xray:UpdateTraceSegmentDestination",
        "application-signals:StartDiscovery",
        "cloudtrail:CreateServiceLinkedChannel"
    ],
    "Resource": "*"
},
{
    "Sid": "CreateServiceLinkedRoleForAppSignals",
    "Effect": "Allow",
    "Action": "iam:CreateServiceLinkedRole",
    "Resource": "arn:*:iam::*:role/aws-service-role/application-
signals.cloudwatch.amazonaws.com/AWSServiceRoleForCloudWatchApplicationSignals"
},
{
    "Sid": "BedrockAgentCoreVendedLogDelivery",
    "Effect": "Allow",
    "Action": "bedrock-agentcore:AllowVendedLogDeliveryForResource",
    "Resource": "*"
}
]
}

```

Vended Logs

- Vended logs are same as application logs in your desired log group.

Vended Spans

- Vended spans are trace logs for requests.

```

import boto3
import json
import time

def enable_observability(resource_arn, resource_id, account_id, region='us-east-1',
enable_xray_spans=False):
    logs_client = boto3.client('logs', region_name=region)

    # Custom header required by service team
    def add_header(model, params, request_signer, **kwargs):
        params['headers']['x-amzn-aqueduct-onboarding'] = 'true'
    logs_client.meta.events.register('before-call.logs', add_header)

    # Create log group
    log_group_name = f'/aws/vendedlogs/bedrock-agentcore/{resource_id}'
    try:
        logs_client.create_log_group(logGroupName=log_group_name)
        print(f"Created log group: {log_group_name}")
    except logs_client.exceptions.ResourceAlreadyExistsException:
        print(f"Log group already exists: {log_group_name}")
    log_group_arn = f'arn:aws:logs:{region}:{account_id}:log-group:{log_group_name}'

```

```

# X-Ray spans setup (resource policy + trace segment destination)
if enable_xray_spans:
    _setup_xray_spans(logs_client, region)

# Delivery sources
print("Creating delivery sources (APPLICATION_LOGS + TRACES)...")
logs_src = logs_client.put_delivery_source(
    name=f"{resource_id}-logs-source", logType="APPLICATION_LOGS",
    resourceArn=resource_arn
)
traces_src = logs_client.put_delivery_source(
    name=f"{resource_id}-traces-source", logType="TRACES",
    resourceArn=resource_arn
)

# Delivery destinations
print("Creating delivery destinations (CWL + XRAY)...")
logs_dst = logs_client.put_delivery_destination(
    name=f"{resource_id}-logs-destination",
    deliveryDestinationType='CWL',
    deliveryDestinationConfiguration={'destinationResourceArn': log_group_arn},
)
traces_dst = logs_client.put_delivery_destination(
    name=f"{resource_id}-traces-destination", deliveryDestinationType='XRAY'
)

# Connect sources to destinations
print("Creating deliveries...")
logs_delivery = logs_client.create_delivery(
    deliverySourceName=logs_src['deliverySource']['name'],
    deliveryDestinationArn=logs_dst['deliveryDestination']['arn'],
)
traces_delivery = logs_client.create_delivery(
    deliverySourceName=traces_src['deliverySource']['name'],
    deliveryDestinationArn=traces_dst['deliveryDestination']['arn'],
)

print(f"Observability enabled for {resource_id}")
return {
    'logs_delivery_id': logs_delivery['delivery']['id'],
    'traces_delivery_id': traces_delivery['delivery']['id'],
}

def _setup_xray_spans(logs_client, region):
    # Allow X-Ray to write spans to CloudWatch Logs
    print("Setting up X-Ray resource policy...")
    logs_client.put_resource_policy(
        policyName="XRaySpansPolicy",
        policyDocument=json.dumps({
            "Version": "2012-10-17",
            "Statement": [{
                "Sid": "XRayAccess",
                "Effect": "Allow",
                "Principal": {"Service": "xray.amazonaws.com"},
                "Action": ["logs:PutLogEvents", "logs:CreateLogGroup",
"logs:CreateLogStream"],
                "Resource": "*",
            }],
        })
    )

    # Switch X-Ray trace segment destination to CloudWatch Logs

```

```

xray_client = boto3.client('xray', region_name=region)
print("Updating X-Ray trace segment destination to CloudWatchLogs...")
try:
    xray_client.update_trace_segment_destination(Destination='CloudWatchLogs')
except xray_client.exceptions.InvalidRequestException as e:
    if "already set to CloudWatchLogs" not in str(e):
        raise
    print(" Already set to CloudWatchLogs, skipping.")

# Wait for ACTIVE status
print("Waiting for X-Ray trace segment destination to become ACTIVE...")
for attempt in range(1, 25):
    resp = xray_client.get_trace_segment_destination()
    destination = resp.get('Destination', {})
    status = destination.get('Status', resp.get('Status', 'UNKNOWN')) if
isinstance(destination, dict) else resp.get('Status', str(destination))
    print(f" Attempt {attempt}/24: {status}")
    if status == 'ACTIVE':
        return
    time.sleep(5)
    raise RuntimeError("X-Ray trace segment destination did not become ACTIVE within
expected time.")

if __name__ == '__main__':
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--enable-xray-spans', action='store_true', default=False)
    args = parser.parse_args()

    resource_arn = "arn:aws:bedrock-agentcore:us-east-1:<account_id>:payment-
manager/<payment-manager-id>"
    resource_id = "<payment-manager-id>"
    account_id = "<account_id>"

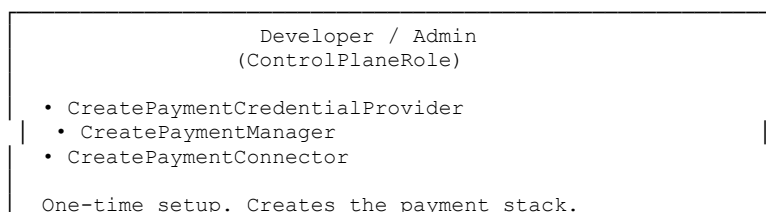
    print(f"Account: {account_id}")
    print(f"Resource ARN: {resource_arn}")
    print(f"Resource ID: {resource_id}\n")

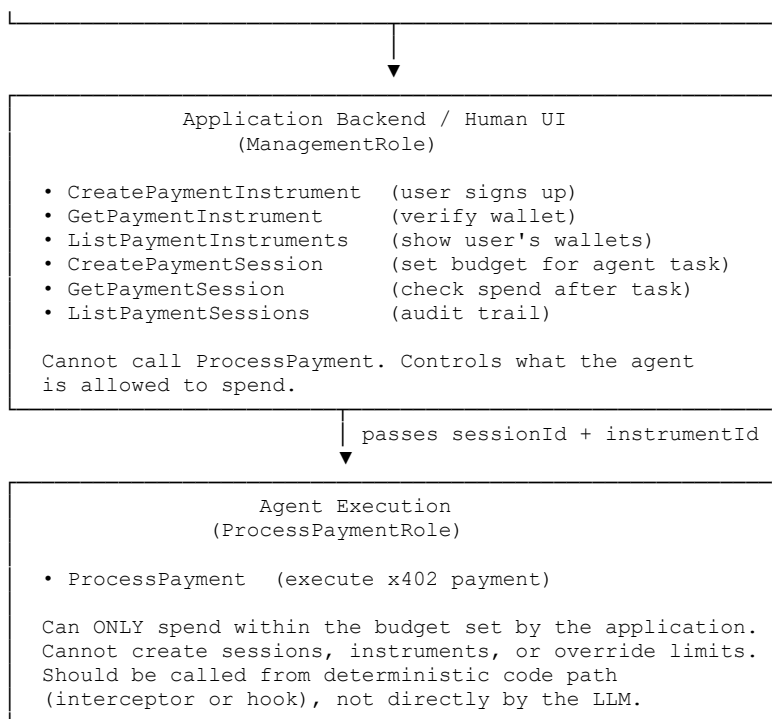
    try:
        result = enable_observability(resource_arn, resource_id, account_id,
enable_xray_spans=args.enable_xray_spans)
        print(f"\nSuccess: {result}")
    except Exception as e:
        print(f"\nFailed: {e}")

```

Reference Architecture

The following diagram shows how the four IAM roles map to the real-world personas in an AgentCore Payments integration:





Agent Instructions

This section provides guidance on how to build payment-enabled agents using AgentCore Payments. The patterns below are based on the reference code sample for Strands agent in `strands-agent/`. Please go through the `readme.md` for more details.

The agent should run under `ProcessPaymentRole` and receive a `paymentSessionId` + `paymentInstrumentId` from the application backend. The agent never creates sessions or instruments: it can only spend within the budget (optional) it was given.

x402 Payment Flow

When the agent encounters a paid endpoint:

1. **Detect 402** The `http_request` tool detects HTTP 402 and extracts payment details:
 - x402 v1: payment details are in the response body JSON
 - x402 v2: payment details are in the `PAYMENT-REQUIRED` header (base64-encoded JSON)
 - The tool returns `x402_payload` the full `accepts[0]` object from the merchant
2. **Call ProcessPayment** Pass the `x402_payload` object as-is to the `process_payment` tool. The `cryptoX402.payload` in the `ProcessPayment` API accepts the raw merchant payment requirement directly; do not parse individual fields. The API signs the transaction using the user's instrument within the session budget.

3. **Retry with proof** Use `http_request_with_payment_header` with the proof from step 2 (Output.paymentOutput.cryptoX402):

- o v1: Build a payload with sends X-PAYMENT: <base64-of the below payload>

```
x402_header = {
  "x402Version": 1,
  "scheme": payment_req["scheme"],
  "network": payment_req["network"],
  "payload": proof["payload"]
}
```

- o v2: sends PAYMENT-SIGNATURE: <base64-json> where the JSON includes an accepted field that must deep-equal the payment requirements from the 402 response

```
payment_signature = {
  "x402Version": 2,
  "resource": payment_req["resource"],
  "accepted": accepts[0], //Selected accepted blob for processPayment
  "payload": proof["payload"]
  "extension": payment_req["resource"]
}
```

4. **Return content** The endpoint validates the proof and returns the paid content.

Setup Steps

Step 1: System Prompt

The following is a tested system prompt template for a payment-enabled agent:

```
You are an AI agent with the ability to make HTTP requests and pay for
paid endpoints using x402 cryptocurrency payments via Amazon Bedrock
AgentCore Payments.
```

Your Capabilities

- Make HTTP requests to any URL using the `http_request` tool
- When you encounter an HTTP 402 (Payment Required) response, pay for access using `process_payment`, then retry with `http_request_with_payment_header`
- You operate under a `ProcessPaymentRole` with a pre-set budget you can only spend within the limits set by the application backend

Payment Flow

When an endpoint returns HTTP 402:

1. The `http_request` tool returns `x402_payload` the merchant's payment requirement
2. Pass `x402_payload` AS-IS to `process_payment` (do NOT parse individual fields)
3. Retry the request using `http_request_with_payment_header` with the proof (Output.paymentOutput.cryptoX402)

Constraints

- Always check the payment amount before paying report it to the user
- You CANNOT create new sessions or instruments
- Process payments from deterministic code paths when possible

```
## Configuration
- Session ID: <injected at runtime>
- Instrument ID: <injected at runtime>
- Network: Base Sepolia (eip155:84532)
- Asset: USDC (0x036CbD53842c5426634e7929541eC2318f3dCF7e)
```

Step 2a: Agent with tools (Strands, LangGraph, CrewAI)

Give the agent the three core tools and the system prompt above. The agent decides when to pay based on 402 responses. See strands-agent/ for a complete working example.

A payment-enabled agent needs three core tools:

Tool	Purpose
http_request	Make HTTP requests. When a 402 is returned, extract the x402 payment details (amount, asset, network, payTo) and return them to the agent. Supports both v1 (body-based) and v2 (PAYMENT-REQUIRED header).
process_payment	Call the ProcessPayment API with SigV4 signing under ProcessPaymentRole. Takes the payment details from the 402 response and returns a signed proof.
http_request_with_payment_header	Retry the original request with the payment proof attached. For v1: X-PAYMENT header. For v2: PAYMENT-SIGNATURE header (base64 JSON with an accepted field that deep-equals the original payment requirements).

Coinbase Bazaar Integration (x402 Client)

We also have a x402 Coinbase Bazaar MCP server with a list of pay-per-use x402 endpoints where you can discover and search for relevant endpoints for their use cases.

The recommended approach is to use the official Coinbase x402 Python client (pip install x402) with an AgentCore adapter. This gives you automatic payment handling — the client detects 402 responses, routes signing through AgentCore ProcessPayment, and retries with the proof automatically.

Prerequisites:

- Python 3.10+
- pip install x402 boto3

Step 1: Create the AgentCore adapter

This adapter implements the x402 SchemeNetworkClient interface and routes signing through AgentCore ProcessPayment instead of a local private key:

```

import uuid
from x402.client import x402ClientSync
from x402.mcp.client import x402MCPCClientSync

class AgentCoreClientScheme:
    """Routes x402 payment signing through AgentCore ProcessPayment."""
    scheme = "exact"

    def __init__(self, dp_client, manager_arn, session_id,
instrument_id, user_id):
        self._dp = dp_client
        self._mgr = manager_arn
        self._sess = session_id
        self._inst = instrument_id
        self._uid = user_id

    def create_payment_payload(self, requirements):
        resp = self._dp.process_payment(
            userId=self._uid,
            paymentManagerArn=self._mgr,
            paymentSessionId=self._sess,
            paymentInstrumentId=self._inst,
            paymentType="CRYPTO_X402",
            paymentInput={"cryptoX402": {"version": "2", "payload": {
                "scheme": requirements.scheme,
                "network": requirements.network,
                "amount": str(requirements.amount),
                "maxAmountRequired": str(requirements.amount),
                "asset": requirements.asset,
                "payTo": requirements.pay_to,
                "maxTimeoutSeconds": requirements.max_timeout_seconds,
                "extra": requirements.extra or {"name": "USDC",
"version": "2"},
            }},
            clientToken=str(uuid.uuid4()),
        )
        if resp.get("status") != "PROOF_GENERATED":
            raise Exception(f"ProcessPayment failed:
{resp.get('status')}")
        return resp["paymentOutput"]["cryptoX402"]["payload"]

```

Step 2: Register the adapter and connect to the Bazaar

```

# Create the adapter with your AgentCore resources
scheme = AgentCoreClientScheme(dp_client, manager_arn, session_id,
instrument_id, user_id)

# Register for both v1 and v2 network names (Bazaar sends v1 format)
x402_client = x402ClientSync()
x402_client.register("eip155:84532", scheme)
x402_client.register("base-sepolia", scheme)
x402_client.register_v1("eip155:84532", scheme)
x402_client.register_v1("base-sepolia", scheme)

```

```
# Wrap your MCP client – payment is now automatic
x402_mcp = x402MCPClientSync(mcp_client, x402_client,
auto_payment=True)
```

Step 3: Call paid tools

```
# The x402 client handles 402 detection, payment, and retry
automatically
result = x402_mcp.call_tool("proxy_tool_call", {
    "toolName": "some_paid_tool",
    "parameters": {}
})
# result.payment_made = True if payment was processed
# result.content contains the paid content
```

Note: The MCP client adapter needs to normalize camelCase fields from the Bazaar (payTo → pay_to, maxTimeoutSeconds → max_timeout_seconds) as the x402 client expects snake_case. See `bazaar-test/x402_agentcore_test.py` for the complete working implementation including this normalization.

See also: `bazaar-test/x402_agentcore_test.py` for a complete end-to-end test that was validated with on-chain settlement on Base Sepolia.

Alternative: Strands Agent Tools

If using the Strands agent framework, three built-in Bazaar tools are available in `strands-agent/agent.py`: `connect_to_bazaar`, `discover_bazaar_tools`, and `call_bazaar_tool`. These handle the full payment flow internally but are framework-specific.

Alternative: HTTP Interceptor (non-MCP endpoints)

For direct HTTP endpoints (not MCP), wrap your HTTP client with an interceptor:

```
def fetch_with_payment(url, session_id, instrument_id):
    resp = requests.get(url)
    if resp.status_code == 402:
        payment_details = extract_x402_details(resp)
        proof = call_process_payment(payment_details, session_id,
instrument_id)
        resp = requests.get(url, headers=build_payment_header(proof,
payment_details))
    return resp
```

Recommended Guardrails

Include these constraints in the agent's system prompt:

- **Report before paying:** The agent should always report the payment amount to the user before executing a payment, unless operating in a fully autonomous mode with pre-approved budget.

- **No escalation:** The agent cannot create new sessions, instruments, or override limits. Only the application backend can do that.
- **Deterministic payment path:** ProcessPayment should ideally be called from a deterministic code path (interceptor or hook) rather than directly by the LLM, to prevent prompt injection from triggering unauthorized payments.

Private Preview (Beta) Guidance

Important Please read before proceeding.

- You may use the Beta Service only for internal testing on your non-production, test accounts. We recommend that you use only fake funds for the Beta Service (i.e. “testnet”). You may not use the Beta Service for or with your external end users, and you may not engage in unlicensed money transmission. You are solely responsible for your use of the Beta Service, including any actions taken by agents.
- Note: These features are provided as a ‘Beta Service’ as defined in the AWS Service Terms. They are governed by your Agreement with AWS and the AWS Service Terms. Before using this Beta Service, please review the Betas and Previews [terms](#). In particular, please note that the Beta Service is confidential and you may not discuss the features, functionality, or documentation related to the Beta Service with any parties that are not authorized by AWS. The Beta Service is subject to change and possible cancellation.

Feedback

Please register your feedback and report any issues to your AWS representative or through the provided feedback channel.

Learn More About AgentCore

AgentCore Payments is part of the broader Amazon Bedrock AgentCore platform. Explore the full suite of capabilities:

Resource	Link
Documentation	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html
Python SDK	https://github.com/aws/bedrock-agentcore-sdk-python/tree/main
Toolkit and Quick Starts	https://aws.github.io/bedrock-agentcore-starter-toolkit/index.html
Sample Labs	https://github.com/aws-labs/amazon-bedrock-agentcore-samples https://github.com/aws-samples/sample-getting-started-with-amazon-agentcore